Double-Entry Ledgers for Backend Engineers

Every example uses orders, refunds and restaurant payouts you already understand. Read top to bottom once, then use as lookup.

The two tables everything runs on

Hold this schema for the whole sheet. Interviews give you something shaped like it.

```
accounts(account_id, name, kind)
```

```
entries(txn_id, account_id, direction, amount_minor, currency, created_at)
```

One money movement = one `txn_id` with two or more entries. `direction` is 'debit' or 'credit'. `amount_minor` is in paise, always positive, always an integer. The one rule: in every transaction, debits equal credits.

Level 1 — an order is paid

"A customer pays Rs. 500. The restaurant gets Rs. 450 and we keep Rs. 50 commission. Post it."

```
Dr PSP clearing         500
  Cr Restaurant payable    450
  Cr Commission revenue    50
```

500 = 450 + 50, so it balances. **PSP clearing** holds money the provider owes us until it settles. **Restaurant payable** is money we owe the restaurant. You already have both ideas in your payout job, just not the names.

Level 2 — a full refund

"The order is cancelled. Refund it."

```
Dr Restaurant payable    450
Dr Commission revenue    50
  Cr PSP clearing         500
```

A refund is a new transaction that mirrors the old one. The original entries stay exactly as they were.

TRAP: Never `UPDATE` or `DELETE` an entry. "Just fix the row" destroys the audit trail, and an auditor or interviewer will stop you there. Corrections are always new entries.

Level 3 — balances are a query, not a column

"What do we owe each restaurant right now?"

```
SELECT account_id,
  SUM(CASE WHEN direction = 'credit' THEN amount_minor ELSE -amount_minor END) AS owed_minor
FROM entries
WHERE account_id IN (SELECT account_id FROM accounts WHERE kind = 'restaurant_payable')
GROUP BY account_id;
```

Liability accounts like payables grow with credits. Asset accounts like PSP clearing grow with debits. Say which sign convention you use before you write the query.

- A cached `balance` column is fine for speed, but it must be updated in the same transaction as the entries. The entries are the truth.

Level 4 — posting exactly once

"The posting job crashed halfway and restarted. How do you avoid posting the order twice?"

```
transactions(txn_id, external_ref UNIQUE, posted_at)
```

Insert the transaction row and all its entries in one database transaction. `external_ref` is the order ID plus the event, for example `order_8812:paid`. A replay hits the unique constraint and becomes a no-op. **[KNOW]** This is your rider-assignment idempotency key.

TRAP: Checking "does a row exist?" and then inserting has a race between two workers. Only the unique constraint is safe.

Level 5 — settlement day

"The provider pays out Rs. 49,000 against Rs. 50,000 of orders. Post it."

```
Dr Bank              49,000
Dr Payment fees       1,000
   Cr PSP clearing     50,000
```

The Rs. 1,000 is the provider's 2% fee (MDR, merchant discount rate). After this, PSP clearing for those orders is zero. A non-zero leftover is a reconciliation break.

Gotchas engineers get burned by

- **Floats**: Rs. 0.10 cannot be stored exactly in a float. Use integer paise, or `NUMERIC` in PostgreSQL.
- **Splitting**: Rs. 100 split three ways is 3,334 + 3,333 + 3,333 paise. Decide who gets the extra paisa, and write the rule down.
- **Day cutoff**: the provider's "day" may end at midnight UTC, yours at midnight IST. Many breaks are just that.
- **Currency**: never sum amounts across currencies. Every entry carries its currency.

5 drills — do cold before any interview

1. Customer pays Rs. 800, commission 15%. Post it. (Dr PSP clearing 800; Cr Restaurant payable 680; Cr Commission revenue 120.)
2. That order gets a Rs. 200 partial refund, and the restaurant bears all of it. Post it. (Dr Restaurant payable 200; Cr PSP clearing 200.)
3. Write the SQL that finds unbalanced transactions. (`GROUP BY txn_id HAVING SUM(signed amount) <> 0`.)
4. Why not keep only a balance column? (No history, no audit, no way to find which event broke it.)
5. A posting worker crashes after writing two of three entries. What did you do wrong? (The entries were not in one database transaction.)

Read next / practice

- Modern Treasury: accounting for developers: debits, credits and normal balances, 20 minutes.
- Square: Books, an immutable double-entry database: how a real company built it, 25 minutes.
- Practice: build the two tables in PostgreSQL, post drills 1 and 2, then run the drill 3 query. One evening.